

Unidad 4

Persistencia de Datos y Buenas Prácticas



Objetivo

Comprender cómo guardar información mediante archivos y bases de datos, integrando POO, MVC y buenas prácticas de desarrollo.



Temas



4.1 Acceso a Archivos en POO



4.2 Bases de Datos y ORM



4.3 Integración de POO, MVC y Acceso a Datos



4.4 Pruebas y Buenas Prácticas



4.1

Acceso a Archivos en POO



¿Qué es?

Permite guardar y recuperar información desde archivos.



Ejemplo de la vida real

Un estudiante guarda sus apuntes en un documento de Word.

Un programa guarda información en archivos TXT, CSV o PDF.

1 Ejemplo Java – Guardar información

```
import java.io.FileWriter;
import java.io.IOException;

public class GuardarArchivo {
    public static void main(String[] args) {
        try {
            FileWriter archivo = new FileWriter("estudiante.txt");
            archivo.write("Ana Perez");
            archivo.close();
            System.out.println("Información guardada correctamente.");
        } catch (IOException e) {
            System.out.println("Error al guardar el archivo.");
        }
    }
}
```



¿Qué hace?

Crea un archivo llamado estudiante.txt y escribe el texto "Ana Perez".

2 Ejemplo Java – Leer información

```
import java.io.File;
import java.util.Scanner;

public class LeerArchivo {
    public static void main(String[] args) {
        try {
            Scanner lector = new Scanner(new File("estudiante.txt"));
            while (lector.hasNextLine()) {
                System.out.println(lector.nextLine());
            }
            lector.close();
        } catch (Exception e) {
            System.out.println("Error al leer el archivo.");
        }
    }
}
```



¿Qué hace?

Abre el archivo estudiante.txt y muestra su contenido en pantalla.

3 Resultado



Archivo creado
estudiante.txt



Salida en consola

```
Ana Perez
```



Idea clave

Los **archivos** permiten almacenar información aunque el programa se cierre.



4.2

Bases de Datos y ORM



¿Qué es una Base de Datos?

Es un sistema que almacena información de forma organizada para que pueda ser usada fácilmente.

Ejemplo: Tabla Estudiantes

id	nombre	edad	carrera
1	Ana Pérez	20	Sistemas
2	Juan López	22	Informática
3	María Gómez	21	Diseño



¿Qué es ORM?

ORM (Object Relational Mapping) es una herramienta que conecta objetos de POO con tablas de una base de datos.



¿Cómo se relacionan los objetos con la base de datos?

1 Clase Java (Objeto)

```
public class Estudiante {
    private int id;
    private String nombre;
    private int edad;
    private String carrera;

    // Constructores, getters y setters...
}
```



2 Tabla SQL

```
CREATE TABLE Estudiantes (
    id INT PRIMARY KEY,
    nombre VARCHAR(50),
    edad INT,
    carrera VARCHAR(50)
);
```

SQL



Objeto en Java

Creamos un objeto Estudiante.



ORM

El ORM convierte ese objeto en un registro.



Base de Datos

El registro se guarda en la tabla Estudiantes.



Recuperación

El ORM convierte los datos de la tabla en objetos Java.



Idea clave

ORM convierte objetos de POO en registros de base de datos y viceversa, de forma automática.



4.3

Integración de POO, MVC y Acceso a Datos

Todo trabaja junto para que el usuario interactúe, los datos se procesen y se guarden correctamente.

Flujo completo del sistema



Ejemplo: Registro de un estudiante



Ejemplo en código (Java)

1 Controlador

```

public class EstudianteController {
    private EstudianteService servicio;

    public void registrarEstudiante(
        String nombre, int edad,
        String carrera) {
        Estudiante e = new Estudiante(
            nombre, edad, carrera);
        servicio.guardar(e);
    }

    // Recibe datos y los envía
    // al servicio
  }

```

2 Modelo (POO)

```

public class Estudiante {
    private String nombre;
    private int edad;
    private String carrera;

    public Estudiante(String n, int e,
        String c) {
        this.nombre = n;
        this.edad = e;
        this.carrera = c;
    }

    // Getters y Setters...
  }

```

3 Servicio + ORM

```

public class EstudianteService {
    private EstudianteRepository repo;

    public void guardar(Estudiante e){
        repo.save(e);
    }

    // El ORM convierte el objeto
    // en un registro en la base
    // de datos
  }

```

4 Registro en la Base de Datos

Tabla: estudiantes

id	nombre	edad	carrera
1	Ana Pérez	20	Sistemas
2	Juan López	22	Informática
3	María Gómez	21	Diseño
4	Luis Torres	23	Sistemas

El estudiante queda almacenado en la BD.



Idea clave

MVC organiza el flujo del sistema, POO estructura los datos y el acceso a datos (ORM) permite guardar y recuperar información de la base de datos.

MVC
Organiza

+

POO
Estructura

+

ORM
Guarda

=

Sistema
completo

4.4

Pruebas y Buenas Prácticas

Probamos para asegurarnos de que todo funcione bien y escribimos código claro y ordenado.

Ejemplo: Probando una función



¿Qué son las pruebas?

Son verificaciones que hacemos para asegurarnos de que el programa funciona como esperamos.

1 Código del programa

```
public class Calculadora {  
    public int suma(int a, int b) {  
        return a + b;  
    }  
}
```



2 Prueba realizada

```
Calculadora calc = new Calculadora();  
int resultado = calc.suma(2, 3);  
System.out.println(resultado);
```



3 Resultado esperado



Si el resultado es 5,
la prueba fue exitosa.

Buenas prácticas en el código

1 Nombres claros

✓ Bueno

```
int totalEstudiantes;  
String nombreCompleto;
```

✗ Evitar

```
int x;  
String n;
```

Usa nombres que describan lo que hacen las variables y funciones.



Comentar cuando sea necesario

✓ Bueno

```
// Calcula el promedio  
int promedio = suma / cantidad;
```

Los comentarios ayudan a entender el código más fácilmente.



Organizar el código por clases

✓ Bueno

```
/Proyecto  
├── Estudiante.java  
├── Profesor.java  
├── Curso.java  
└── Main.java
```

Mantener el código organizado facilita encontrar y usar las clases.



Código limpio y simple

✓ Bueno

```
if (edad >= 18) {  
    System.out.println("Mayor de edad");  
}
```

✗ Evitar código complicado

```
if(edad>=18){System.out.println("Mayor");}  
else{System.out.println("Menor");}
```

Código simple es más fácil de leer, entender y mantener.



Resumen final



Archivos

Guardan información en el disco.



Base de datos

Almacena grandes cantidades de datos.



ORM

Conecta objetos y tablas.



MVC

Organiza el sistema en partes.



Pruebas

Verifican que el programa funcione correctamente.



Buenas prácticas

Hacen el código más claro, seguro y mantenible.

4.5

Resumen y Conclusión

La persistencia de datos y las buenas prácticas aseguran sistemas confiables, organizados y fáciles de mantener.



1. Archivos

Guardan información en el disco para que no se pierda.



2. Base de Datos y ORM

Almacenan grandes cantidades de datos y conectan con objetos.



3. Integración MVC

Organiza el sistema en capas y asegura un flujo ordenado de información.



4. Pruebas y Buenas Prácticas

Verifican que todo funcione correctamente y facilitan el mantenimiento.

Ejemplos rápidos de todo lo visto

1 Archivos (Guardar y leer)



```
// Guardar
FileWriter archivo =
    new FileWriter("datos.txt");
archivo.write("Hola Mundo");
archivo.close();

// Leer
Scanner lector = new Scanner
    (new File("datos.txt"));
System.out.println(lector.nextLine());
```

Se crea un archivo y luego se lee su contenido.

2 Base de Datos y ORM



```
// Objeto Java
class Estudiante {
    int id;
    String nombre;
    int edad;
}

// ORM guarda en la tabla
Estudiante e = new Estudiante();
e.setNombre("Ana Pérez");
e.setEdad(20);
repository.save(e);
```

El ORM convierte el objeto en un registro en la base de datos.

3 Integración MVC



Usuario
Interactúa

Vista (View)
Muestra información
y recibe datos

Controlador (Controller)
Recibe datos y controla
el flujo

Modelo (Model)
Usa clases y ORM para
guardar en la BD

Base de Datos
La información se almacena
y recupera

El MVC organiza el flujo y el ORM
guarda la información.

4 Pruebas y Buenas Prácticas

Prueba de una función

```
int suma(int a, int b) {
    return a + b;
}

// Prueba
int resultado = suma(2, 3);
System.out.println(resultado); // 5
```

Buenas prácticas

- ✓ Nombres claros y descriptivos
- ✓ Comentar cuando sea necesario
- ✓ Código limpio y simple
- ✓ Organizar por clases y paquetes
- ✓ Probar el código regularmente



Las pruebas aseguran que el código
funcione bien. Las buenas prácticas
lo mantienen claro y seguro.



En resumen



Archivos
Guardan
información.



Base de Datos
Almacenan grandes
cantidades.



ORM
Conecta objetos
y tablas.



MVC
Organiza el
sistema.



Pruebas
Verifican que todo
funcione.



Buenas Prácticas
Hacen el código
mejor y durable.



Todo junto permite crear sistemas
confiables, seguros y fáciles de
mantener en el tiempo.



GRACIAS

— POR TU ATENCIÓN —

...

